
Software Design Specification

for

LLMUSE

Version 1.0 approved

Prepared by Team 4

2025. 11. 30.

Table of Contents

Table of Contents	ii
Revision History	iii
1. Purpose.....	1
1.1 Readership	1
1.2 Scope.....	1
1.3 Objective	1
1.4 Document Structure	1
2. Introduction.....	2
2.1 Objectives	2
2.2 Applied diagrams	2
2.2.1 Used tools	
2.2.2 Use case diagram	
2.2.3 Sequence diagram	
2.2.4 Class diagram	
2.2.5 Context diagram	
2.2.6 Entity relationship diagram	
2.2.7 Project scope	
2.2.8 References	
3. System Architecture - Overall	3
3.1 Objectives	3
3.2 System organization.....	3
3.2.1 System diagram	
3.3 Use case diagram	4
4. System Architecture - Frontend	5
4.1 Objectives	5
4.2 Main page interfaces.....	5
4.2.1 Attributes	
4.2.2 Methods	
4.2.3 Class diagram	
4.2.4 Sequence diagram	
4.3 Search and result components.....	7
4.3.1 Attributes - SearchBar	
4.3.2 Methods - SearchBar	
4.3.3 Attributes - ResultList	
4.3.4 Methods - ResultList	
4.3.5 Class diagram	
4.3.6 Sequence diagram	
5. System Architecture - Backend	8
5.1 Objectives	8
5.2 Subcomponents	8
5.2.1 Control component - Chat Module	
5.2.2 Query analysis component	
5.2.3 Query reinforcement component	
5.2.4 Search component	
5.2.5 LLM response component	
5.2.6 Sequence diagram	
6. Protocol Design.....	10
6.1 Objectives	10
6.2 Restful API and JSON	10
6.3 TLS and HTTPS	10

6.4	Chat interaction	10
6.5	Chat Sesion Management	11
6.5.1	Create chat session	
6.5.2	Get session list	
6.5.3	Sequence diagram	
6.5.5	Update and delete session	
7.	Database Design	13
7.1	Objectives	13
7.2	ER diagrams.....	13
7.2.1	User	
7.2.2	ChatHistory	
7.2.3	BoxOffice	
7.2.4	Movie	
7.2.5	Performance	
7.3	Relational Schema	16
8.	Testing Plan	16
8.1	Objectives	16
8.2	Testing policy	16
8.2.1	Development testing	
8.2.2	Integration and system testing	
8.2.3	Performance testing	
8.2.4	Release testing	
8.2.5	User testing	
8.2.6	Test case	
9.	Development Plan	17
9.1	Objectives	17
9.2	Frontend environment.....	17
9.2.1	Nest.js	
9.2.2	Typescript	
9.2.3	Hypertext markup language	
9.2.4	Tailwind css	
9.2.5	Material UI	
9.2.6	Axios	
9.2.7	Socket.io - client	
9.3	Backend environments.....	19
9.3.1	ChromaDB	
9.2.2	Nest.js	
9.2.3	Github	
9.4	Constraints	20
9.5	Assumptions and dependencies	20

Revision History

Name	Date	Reason For Changes	Version
Design Specification	25.11.30.	First Version	1.0

1. Purpose

1.1 Readership

이 문서는 다음과 같은 독자들을 위해 만들어졌다. 본 시스템의 개발자들이다. 시스템 개발자는 크게 프론트엔드 개발자와 백엔드 개발자로 나뉜다. 그리고 소프트웨어 공학 개론 수업의 교수, 조교이다.

1.2 Scope

개발 과정에서 프론트엔드와 백엔드 개발자가 참고하기 위한 시스템 아키텍처와 구조를 설명하고, 각 요소에서 사용되는 기술들과 데이터베이스의 구조를 설명하고 있다. 또한 테스트 과정과 전반적인 개발 과정에서의 계획을 포함하고 있다.

1.3 Objective

이 문서는 영화 추천 시스템(이하 LLMUSE)의 기술적 설계에 대한 설명이다. 이 문서는 실습 프로그램의 구현의 기반이 되는 소프트웨어 프론트엔드, 백엔드, Database 측면에서의 설계를 정의한다. 모든 설계는 앞서 제작된 Software Requirements Specification 문서의 요구 사항을 기반으로 작성되었다.

1.4 Document structure

- 1) Preface: 본 문서의 목적, 예상 독자 및 문서의 구조에 대해 설명한다.
- 2) Introduction: 본 문서를 작성하는데 사용된 도구들과 다이어그램들, 참고 자료들에 대해 설명한다.
- 3) Overall System Architecture: 시스템의 전체적인 구조를 Context diagram, Use case diagram, Sequence diagram을 이용하여 서술한다.
- 4) System Architecture – Frontend: Frontend 시스템의 구조를 Class diagram, Sequence diagram을 이용하여 서술한다.
- 5) System Architecture – Backend: Backend 시스템의 구조를 Sequence diagram, Class diagram을 이용하여 서술한다.
- 6) Protocol Design: 클라이언트와 서버의 커뮤니케이션을 프로토콜 디자인을 서술한다.
- 7) Database Design: 시스템의 Database Requirements를 기반으로 ER Diagram, Relation Schema를 이용하여 시스템의 데이터베이스 디자인을 서술한다.
- 8) Testing Plan: 시스템을 위한 테스트 계획을 서술한다.
- 9) Development Plan: 시스템의 구현 계획 및 구현을 위한 개발 도구, 라이브러리 등의 개발 환경을 설명한다.

2. Introduction

2.1 Objectives

이번 챕터에서는 제안한 시스템의 설계에 사용된 다이어그램, 톨에 대해 설명하고 개발 범위에 대해 설명한다.

2.2 Applied diagrams

2.2.1 Used tools

Microsoft Powerpoint 발표용 자료 제작 프로그램인 Powerpoint 를 통해 기본적인 도형이나 아이콘을 생성할 수 있는 도구로 다이어그램이 작성되었다.

2.2.2 Use case diagram

Use case Diagram 은 시스템에서 제공해야 하는 기능이나 서비스를 명세화한 다이어그램이다. 사용자와 use cases 간의 관계를 보여주며, 사용자 - 시스템 간 상호작용을 표현한다.

2.2.3 Sequence diagram

Sequence Diagram 은 시간순서로 행동별로 어떤 객체와 어떻게 상호작용을 하는지 표현하는 Diagram 이다. Event Diagram 이라고도 부르는데, 시나리오와 관련된 객체와 시나리오의 기능 수행에 필요한 객체 간에 교환되는 메시지를 순서대로 표현한다.

2.2.4 Class diagram

Class Diagram 은 시스템을 구성하는 클래스, 그 속성, 기능 및 객체들 간의 관계를 표현하여 시스템의 정적인 부분을 보여준다. 이 시스템에서는 크게 사용자(student), 클라이언트, 서버가 있다고 봤다. 이 다이어그램은 실제로 구현될 소스코드와는 다를 수 있으며 의미나 해석 또한 경우에 따라 달라질 수 있다.

2.2.5 Context diagram

Context Diagram 은 Data Flow Diagram 에서 가장 높은 수준의 다이어그램으로, 시스템 전체와 외부 요인의 입력 및 출력을 보여준다. 시스템과 해당 시스템과 상호작용할 수 있는 모든 외부의 엔티티들을 나타내며, 그 사이의 정보의 흐름을 표현한다. 엔지니어 또는 개발자가 사용하기보다는 프로젝트의 이해 관계자가 사용한다.

2.2.6 Entity relationship diagram

ER Diagram 은 구조화된 데이터와 그들 간의 관계를 사람이 이해할 수 있는 형태로 표현하는 다이어그램이며, 현실에서의 요구사항들을 이용한 데이터베이스 설계과정에서 활용된다. 해당 다이어그램은 Entity, Attribute, Relationship 으로 구성된다. 이 다이어그램을 통해 데이터베이스의 논리적 구조를 설명한다.

2.2.7 Project scope

LLMUSE 는 사용자들의 영화 및 공연 취향을 참고하여 다른 영화를 추천해주고, 이를 이용해서 실제적인 사용으로 이어질 수 있도록 만들어졌다.

2.2.8 References

IEEE Std 830-1998 IEEE Recommended Practice for Software Requirements Specifications, In IEEEExplore Digital Library
<http://ieeexplore.ieee.org/Xplore/guesthome.jsp>

3. System Architecture – Overall

3.1 Objectives

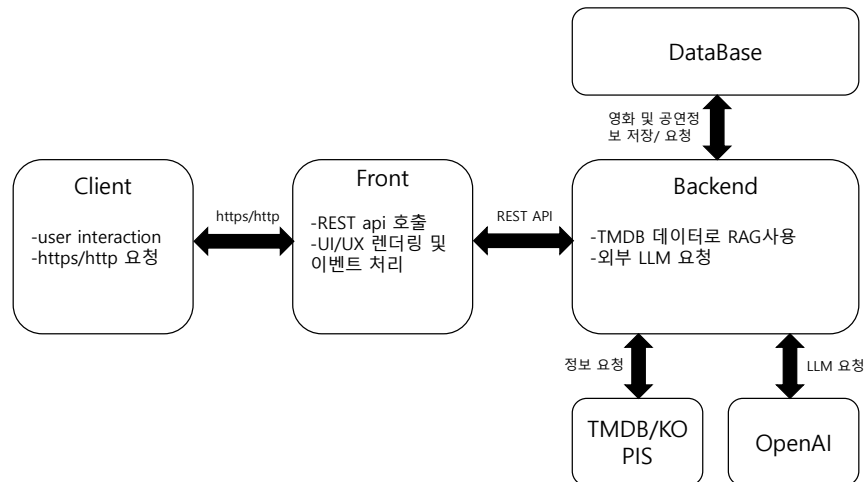
이 챕터에서는 프론트엔드 설계에서 백엔드 설계, 그리고 AI/Data Layer 에 이르는 프로젝트 어플리케이션의 전체적인 시스템 구성과 데이터 흐름에 대해 설명한다.

3.2 System Organization

LLMUSE 시스템은 Next.js 프론트엔드와 Nest.js 백엔드, 그리고 ChromaDB 및 SQLite 로 구성된 데이터 계층이 상호작용하는 클라이언트-서버 구조를 기반으로 설계되었다. 시스템의 각 구성 요소는 RESTful API 와 WebSocket 을 통해 JSON 형식의 데이터를 교환하며 유기적으로 동작한다.

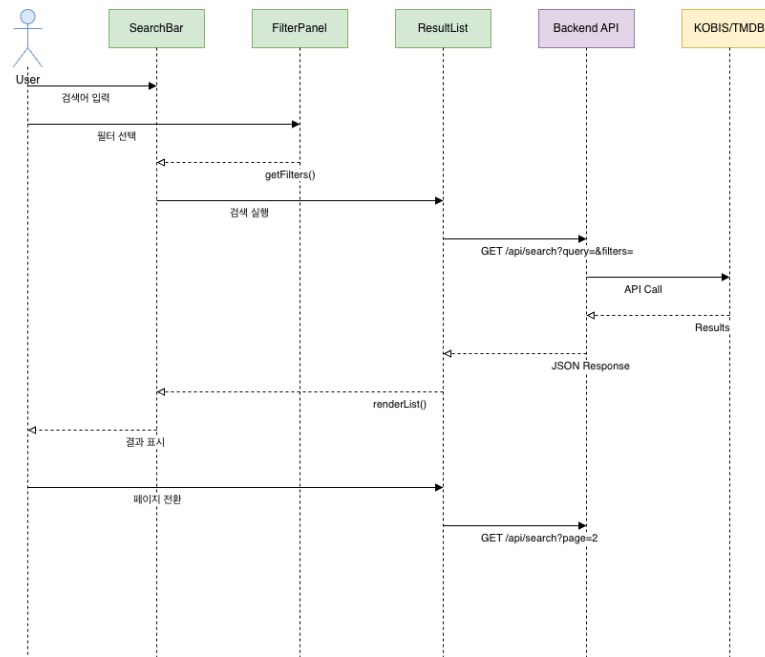
사용자 인터페이스 계층인 프론트엔드는 사용자의 모든 상호작용을 담당하며, 자연어 검색 입력부터 결과 조회, 상세 정보 탐색에 이르는 전반적인 UI/UX 를 제공한다. 또한 백엔드 API 와의 통신을 통해 영화 추천 및 검색 결과를 화면에 렌더링하고, WebSocket 기반의 실시간 채팅 인터페이스를 지원한다.

시스템의 핵심 제어 계층인 백엔드는 프론트엔드로부터 전달된 자연어 질의와 동작 요청을 수신하여 적절한 서비스 로직으로 라우팅하는 역할을 수행한다. 특히 RAG(Retrieval-Augmented Generation) 파이프라인을 통해 사용자의 질의를 LLM 으로 분석하여 벡터화하고, ChromaDB 에서 관련 컨텍스트를 조회한 뒤 이를 OpenAI LLM 과 결합하여 최종 답변을 생성·반환한다. 아울러 백엔드는 SQLite 와 ChromaDB 를 활용해 데이터의 무결성과 일관성을 관리하며, KOBIS 그리고 TMDB API 로부터 최신 영화 및 공연 데이터를 주기적으로 수집·가공하는 ETL 프로세스를 수행하여 데이터베이스를 항상 최신 상태로 유지한다



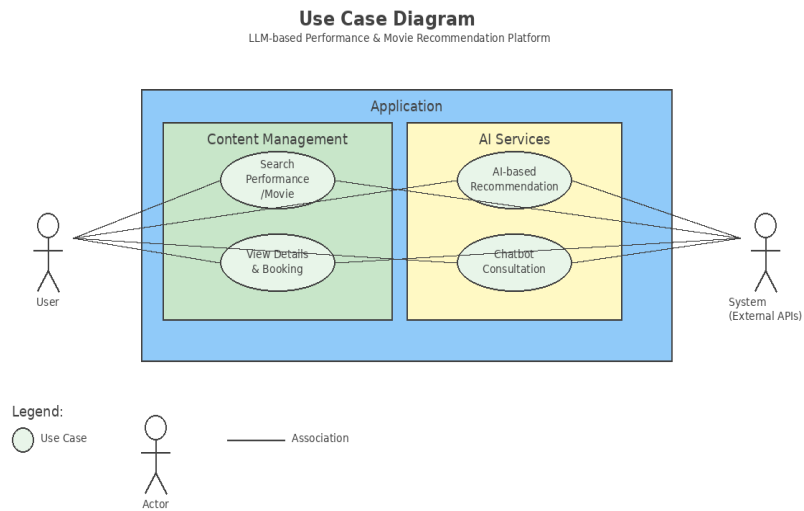
[Fig 3.1] System organization

3.2.1 System diagram



[Fig 3.2] System diagram, Overall

3.3 Use Case Diagram



[Fig 3.3] Use case diagram

4. System Architecture - Frontend

4.1 Objectives

이 장에서는 LLMUSE 의 프론트엔드 시스템 구조, 주요 컴포넌트, 그리고 사용자 인터페이스 설계를 설명한다.

4.2 Main Page Interfaces

메인 페이지 인터페이스는 사용자가 LLMUSE 와 상호작용하는 핵심 진입점이다. 챗봇 기반의 대화형 인터페이스를 통해 사용자는 영화 추천 및 정보를 자연어로 질의하고, 실시간으로 응답받을 수 있다.

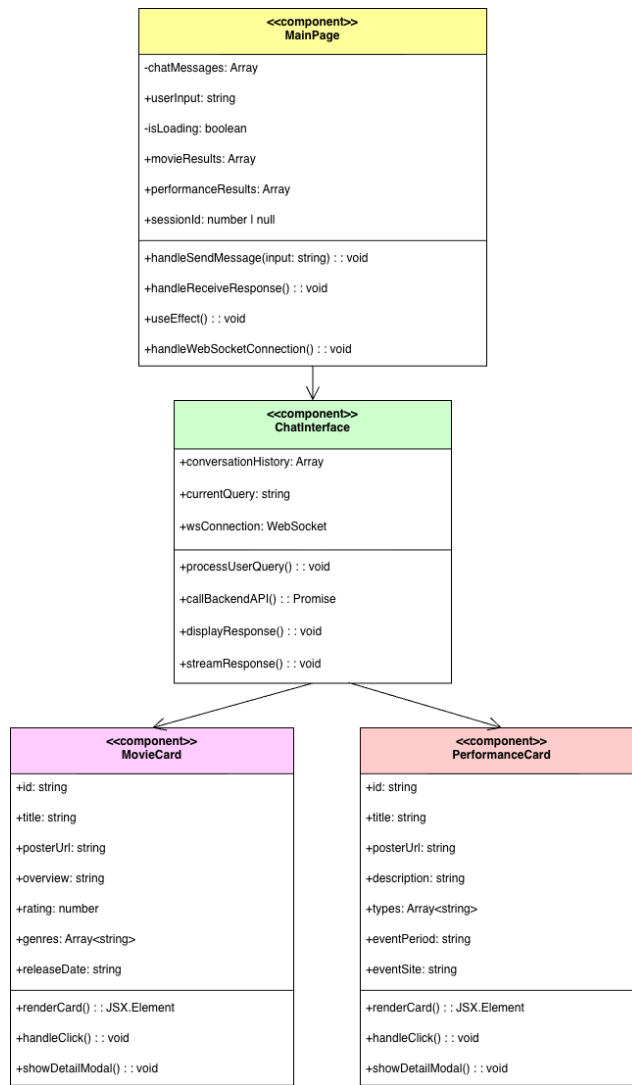
4.2.1 Attributes

Name	Type	Description
chatMessages	Array	대화 기록을 저장하는 배열
userInput	String	사용자의 현재 입력 텍스트
isLoading	Boolean	API 응답 대기 상태
movieResults	Array	LLM 이 추천한 영화 목록
sessionId	String	WebSocket 세션 식별자

4.2.2 Methods

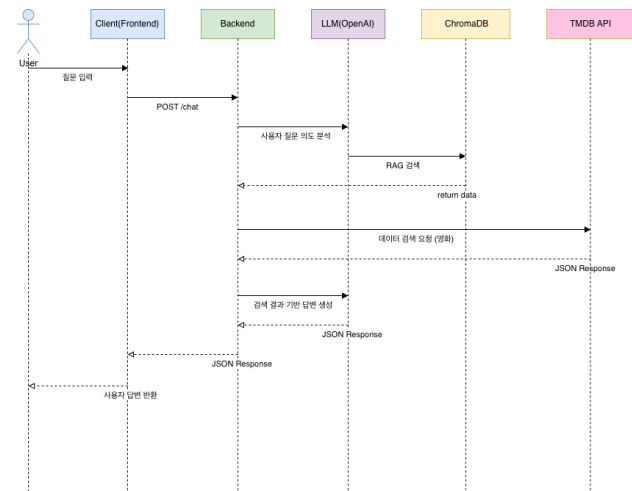
Name	Type	Description
sendMessage()	Void	사용자 메시지를 Backend 로 전송
receiveResponse()	Void	Backend 로부터 응답 수신 및 UI 업데이트
displayMovieCard()	Void	영화 정보를 카드 형태로 렌더링
handleWebSocketConnection()	Void	실시간 통신 연결 관리

4.2.3 Class diagram



[Fig 4.1] Sequence diagram, Frontend

4.2.4 Sequence diagram



[Fig 4.2] Sequence diagram, Frontend

4.3 Search and Result Components

검색 및 결과 컴포넌트는 사용자가 특정 조건으로 영화를 검색하고, 필터링된 결과를 확인할 수 있도록 지원한다. 장르, 개봉일, 평점 등 다양한 필터를 제공하여 정밀한 검색이 가능하다.

4.3.1 Attributes - SearchBar

Name	Type	Description
SearchQuery	String	사용자 검색어
Filters	Object	선택된 필터 조건(장르, 날짜, 평점)
Suggestions	Array	자동완성 제안 목록

4.3.2 Methods - SearchBar

Name	Type	Description
handleSearch()	Void	검색 실행
applyFilters()	Void	필터 조건 적용
validateInput()	Boolean	입력 유효성 검사
fetchSuggestions()	Void	자동완성 데이터 요청

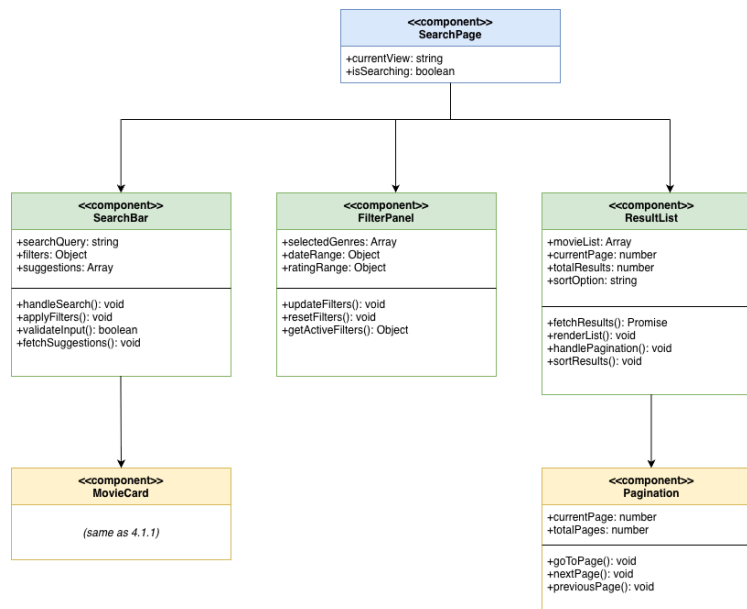
4.3.3 Attributes - ResultList

Name	Type	Description
movieList	Array	검색 결과 영화 목록
currentPage	Number	현재 페이지 번호
totalResults	Number	전체 결과 개수
sortOption	String	정렬 기준(인기, 최신, 평점)

4.3.4 Methods - ResultList

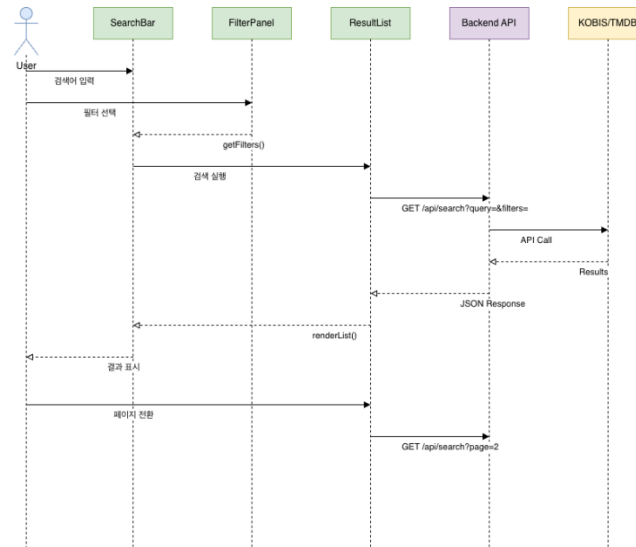
Name	Type	Description
fetchResults()	Promise	Backend 에서 검색 결과 가져오기
renderList()	Void	결과를 그리드 형태로 렌더링
handlePagination()	Void	페이지 전환 처리
sortResults()	Void	정렬 기준 변경

4.3.5 System diagram



[Fig 4.3] System diagram, Components

4.3.6 Sequence diagram



[Fig 4.4] Sequence diagram, Components

5. System Architecture - Backend

5.1 Objectives

본 챕터에서는 백엔드 시스템의 구조 및 컴포넌트들에 대해서 기술한다.

5.2 Subcomponents

백엔드 시스템은 데이터의 흐름과 역할에 따라 아래의 Component 들로 구분할 수 있다.

5.2.1 Control component – Chat Module

역할: 시스템의 메인 컨트롤 타워로, 클라이언트의 요청(POST /chat)을 수신하고 전체 비즈니스 로직의 흐름을 제어한다.

기능:

- Session Management: 요청에 포함된 session_id 를 기반으로 대화 정보를 가져와 적용한다.
- Pipeline Execution: 분석 > 정제 > 검색(영화/공연) > 생성 단계의 순차적 실행을 조율한다.
- Transaction Management: 최종 생성된 답변과 추천 데이터를 클라이언트에게 반환한다.

5.2.2 Query analysis component

역할: 사용자의 불명확한 자연어 입력을 LLM(OpenAI)를 통해 질문을 분석하여 구조화된 데이터로 변환한다.

기능:

- Intent Classification: 질문의 의도를 완전 검색, 관련 검색, 조건 검색 등으로 분류한다.
- Keyword Extraction: 질문 내에서 영화 제목, 공연명, 배우, 장르 등 핵심 엔티티를 추출하여 JSON 형태로 반환한다.

5.2.3 Query refinement component

역할: 추출된 키워드의 오타를 수정하고, 데이터베이스에 저장된 표준 명칭으로 정규화한다.

기능: ChromaDB의 벡터 유사도 검색을 활용하여 벡터 간의 거리를 계산하여 가장 유사한 표준어로 보정한다.

5.2.4 Search component

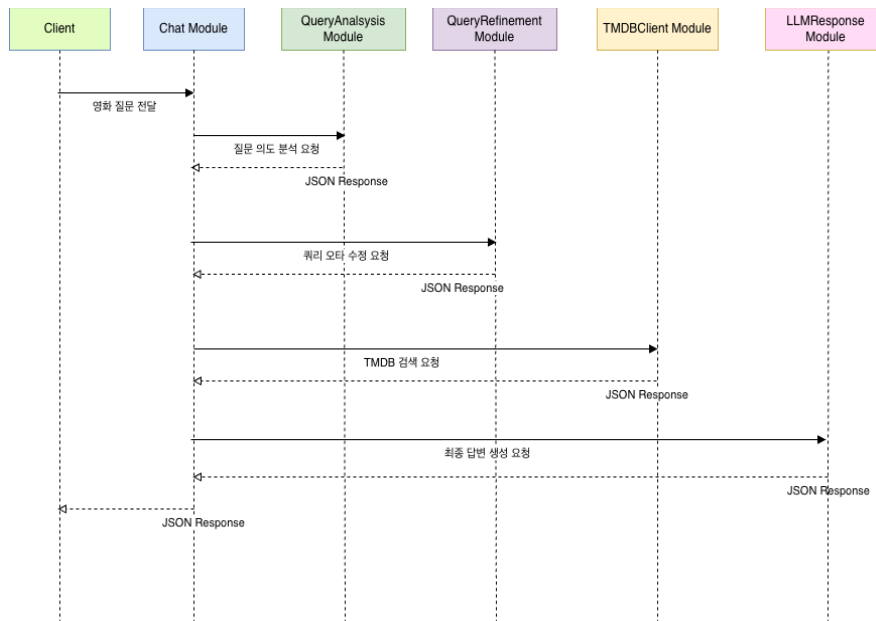
도메인의 특성에 따라 두 가지의 서로 다른 검색 전략을 수행하는 컴포넌트이다.

- a. Movie Search Component
 - i. Data Source: TMDB API
 - ii. 기능: 영화 도메인 검색을 담당한다. 정제된 키워드를 바탕으로 TMDB의 Search, Discover, Credits API를 호출하여 질문 의도에 맞는 영화 데이터를 수집한다.
- b. Performance Search Component
 - i. Data Source: ChromaDB (performance collection)
 - ii. 기능: 공연 도메인 검색을 담당한다. 공연 데이터는 사전에 벡터화되어 저장되어 있으므로, 쿼리의 의미적 유사성을 기반으로 적합한 공연 정보를 추출한다. 날짜, 장소 등의 메타데이터 필터링을 병행하여 정확도를 높인다.

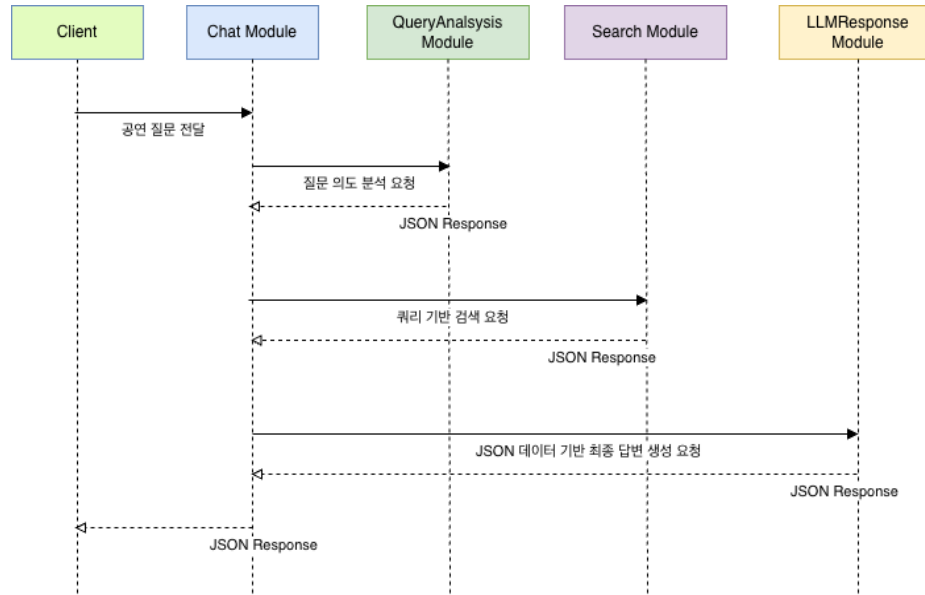
5.2.5 LLM response component

역할: 수집된 검색 결과와 대화 문맥을 종합하여 최종적인 자연어 답변을 생성한다.
기능: 시스템 프롬프트와 검색된 데이터(JSON), 사용자의 질문을 종합하여 LLM의 최종 답변을 요청한다.

5.2.6 Sequence diagram



[Fig 5.1] Sequence diagram, Movie



[Fig 5.2] Sequence diagram, Performance

6. Protocol Design

6.1 Objectives

본 챕터는 서버와 클라이언트 간 어떤 프로토콜로 상호작용하는지 기술하고, 데이터 교환을 위한 포맷과 각 인터페이스가 어떻게 정의되는지 기술한다.

6.2 Restful API and JSON

본 시스템은 REST(Representational State Transfer) 아키텍처 스타일을 따른다. 클라이언트와 서버는 HTTP 프로토콜을 통해 통신하며, Resource 는 URL 로 식별된다. 데이터 형식은 JSON 을 사용한다. JSON 형식은 사람이 읽고 쓰기가 쉬우며, 기계가 파싱하고 생성하기에도 용이하여, 본 시스템의 영화 메타데이터 구조를 표현하는데 적합하다.

6.3 TLS and HTTPS

TLS(Transport Layer Security)는 웹사이트와 브라우저 사이(또는 서버와 서버 사이)에 전송되는 데이터를 암호화하는 표준 기술이다. HTTPS(HyperText Transfer Protocol over Transport Layer Security)는 기존의 HTTP 를 TLS 위에서 패키징하여 강화된 보안을 제공하는 버전이다. 본 시스템은 사용자 질문 및 개인화된 추천 기록 보호를 위해 모든 통신에 HTTPS 를 권장한다.

6.4 Chat Interaction

- 1) Request

Attribute	Detail	
Protocol	HTTPS	
Method	POST	
Endpoint	/chat	
Request Body	User_id	User identifier
	Session_id	Session ID
	Question	User natural language query

[Table 6.1] Chat Request

2) Response

Attribute	Detail	
Protocol	HTTPS	
Success Code	201 Created	
Failure Code	HTTP error code = 400 (Bad request), 500 (Internal Server Error)	
Success Response Body	Session_id	Current Session ID
	Question	Echoed user question
	Answer	AI generated answer
	Movies / performances	List of Movie, performance objects
Failure Response Body	Message	Error description message
	Error	Error type or code

[Table 6.2] Chat Respons

6.5 Chat Session Management

사용자는 주제별로 여러 개의 대화방을 생성하여 관리할 수 있다. 이 인터페이스는 대화방의 생성, 조회, 수정, 삭제 기능을 제공하게 된다.

6.5.1 Create Chat Session

1) Request

Attribute	Detail	
Protocol	HTTPS	
Method	POST	
Endpoint	/chat/sessions	
Request Body	User_id	User identifier
	Title	Session title

[Table 6.3] Create Session Request

2) Response

Attribute	Detail	
Protocol	HTTPS	
Success Code	201 Created	
Failure Code	HTTP error code = 400 (Bad request), 500 (Internal Server Error)	
Success Response Body	Session_id	Generated Session ID
	Title	Session title
	Created_at	Created timestamp
Failure Response Body	Message	Error description message
	Error	Error type or code

[Table 6.4] Create Session Response

6.5.2 Get session list

1) Request

Attribute	Detail	
Protocol	HTTPS	
Method	GET	
Endpoint	/chat/sessions	
Request Body	User id	User identifier

[Table 6.5] Get Session List Request

2) Response

Attribute	Detail	
Protocol	HTTPS	
Success Code	200 OK	
Failure Code	HTTP error code = 400 (Bad request), 500 (Internal Server Error)	
Success Response Body	Sessions	List of Session Objects
	Id	Session ID
	Title	Session title
	Updated at	Last activity time stamp
Failure Response Body	Message	Error description message
	Error	Error type or code

[Table 6.6] Create Session Response

6.5.3 Update and delete session

대화방의 제목을 수정하거나, 더 이상 필요 없는 대화방을 삭제한다

1) Request

Attribute	Detail	
Protocol	HTTPS	
Method	PATCH / DELETE	
Endpoint	/chat/sessions/(session id)	
Request parameter	Session id	Session ID
Request body	Title	New Title

[Table 6.7] Update /Delete Session Request

2) Response

Attribute	Detail	
Protocol	HTTPS	
Success Code	200 OK / 204 No Content	
Failure Code	HTTP error code = 400 (Bad request), 500 (Internal Server Error)	
Success Response Body	Sessions	List of Session Objects
	Id	Session ID
	Message	Success confirmation message
Failure Response Body	Message	Error description message
	Error	Error type or code

[Table 6.8] Update /Delete Session Response

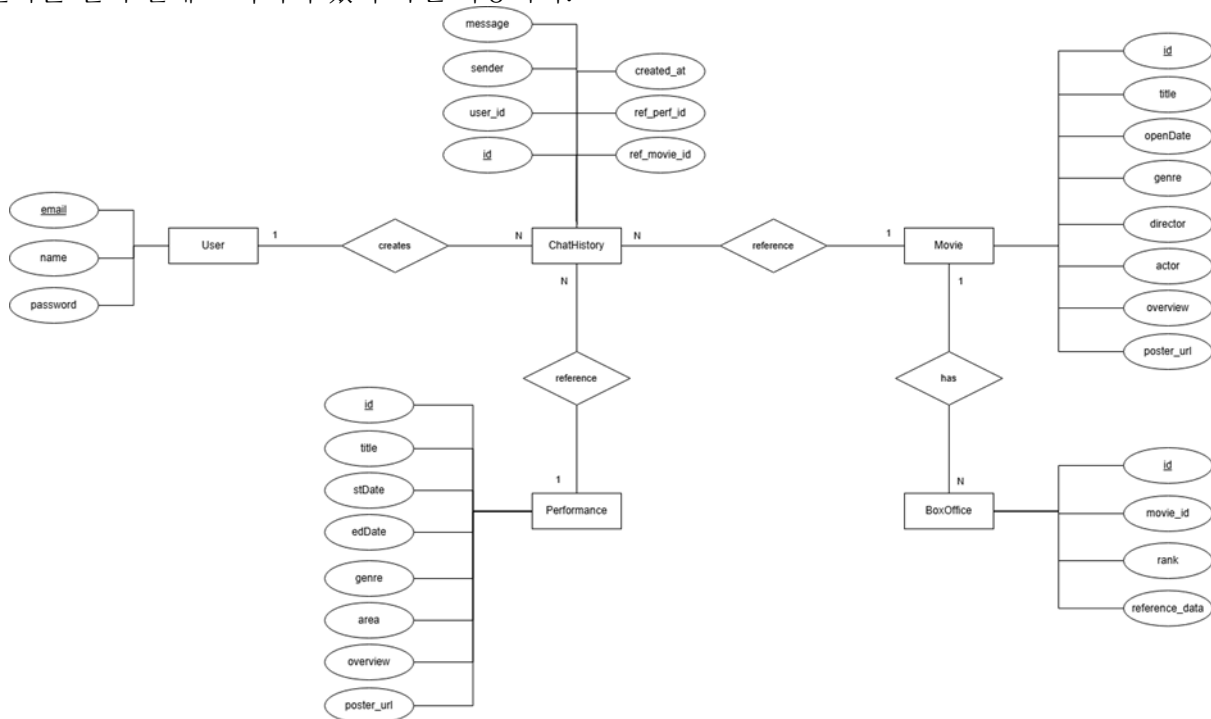
7. Database Design

7.1 Objectives

7 장에서는 시스템 데이터 구조와 이러한 구조가 데이터베이스로 어떻게 구현되었는지에 대해 설명한다. 먼저 ER 다이어그램(Entity Relationship diagram)을 통해 entity와 그 관계를 식별한다. 그런 다음 관계형 스키마를 작성한다.

7.2 ER diagrams

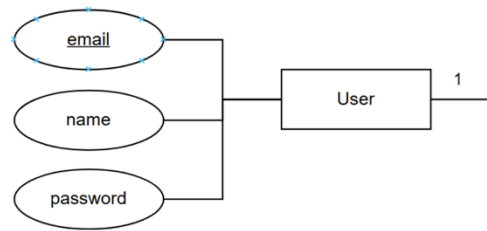
본 어플리케이션 시스템은 총 5 가지 entity로 이루어져 있다: User, ChatHistory, BoxOffice, Movie, Performance. ER-Diagram은 entity 간의 관계, 그리고 entity와 attribute의 관계를 다이어그램으로 설명한다. 각 entity의 primary key는 밑줄로 표시되어 있다. 각 entity마다 대응되는 개수는 entity를 연결하는 선 주변에 표기되어 있어 확인 가능하다.



[Fig 7.1] ER Diagram, Entity

7.2.1 User

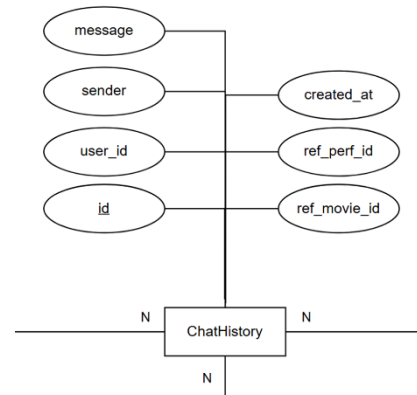
User Entity는 어플리케이션 사용자에게 해당한다. User Entity는 email, name, password를 attribute로 가지며, email이 primary key이다. 이 attribute들은 어플리케이션 회원 가입 때 사용자가 기입한 정보이다.



[Fig 7.2] ER Diagram, User

7.2.2 ChatHistory

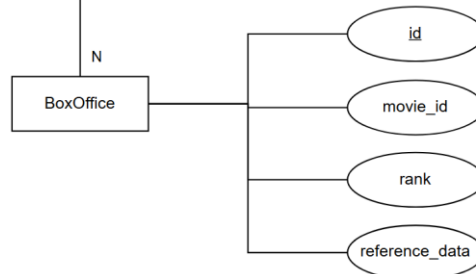
ChatHistory Entity 는 사용자와 AI 챗봇 간에 주고받은 개별 메시지 기록에 해당한다. ChatHistory Entity 는 id, user_id, sender, message, created_at, ref_movie_id, ref_perf_id 를 attribute 로 가지며, id 가 primary key 이다. 이 attribute 들 중 user_id 는 메시지를 보낸 사용자를 식별하는 외래키이고, sender 는 발신자(사용자 또는 AI)를 구분하며, message 는 실제 대화 내용을 담고 있다. created_at 은 대화의 순서를 정렬하기 위한 생성 일시이며, ref_movie_id 와 ref_perf_id 는 AI 가 추천한 영화나 공연 정보를 연결하여 사용자에게 예매 링크 등을 제공하기 위해 사용되는 선택적 정보이다.



[Fig 7.3] ER Diagram, ChatHistory

7.2.3 BoxOffice

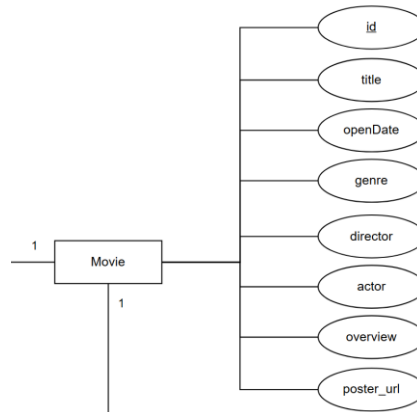
BoxOffice Entity 는 애플리케이션의 메인 화면 우측 사이드바에 표시될 일별 박스오피스 순위 정보를 관리하는 데 해당한다. BoxOffice Entity 는 id, movie_id, rank, reference_date 를 attribute 로 가지며, id 가 primary key 이다. 이 attribute 들 중 movie_id 는 순위 정보가 어떤 영화에 대한 것인지 식별하기 위해 Movie Entity 를 참조하는 외래키이고, rank 는 해당 일자의 1 위부터 10 위까지의 순위 정보를 담고 있다. reference_date 는 순위의 기준이 되는 날짜를 기록하여 매일 갱신되는 박스오피스 데이터를 관리하고 과거 순위 조회도 가능하게 한다.



[Fig 7.4] ER Diagram, BoxOffice

7.2.4 Movie

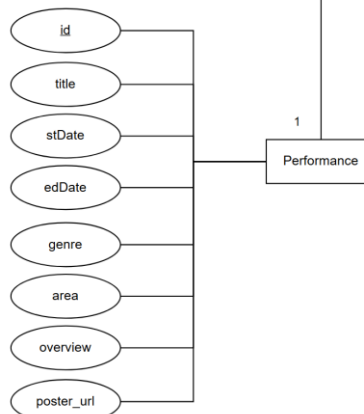
Movie Entity 는 애플리케이션에서 사용자에게 제공할 영화의 상세 정보를 저장하고 RAG 기반 검색을 지원하기 위한 핵심 개체이다. Movie Entity 는 id, title, openDate, genre, director, actor, overview, poster_url 을 attribute 로 가지며, id 가 primary key 이다. 이 attribute 들 중 title 은 영화의 제목을, openDate 는 개봉일을 나타내며, genre, director, actor 는 각각 영화의 장르, 감독, 출연 배우 정보를 저장하여 사용자가 영화를 파악하는 데 필요한 메타데이터를 구성한다. 특히 overview 는 영화의 줄거리를 담고 있어 RAG 기술을 통해 사용자의 질문 맥락에 맞는 영화를 검색하고 추천 답변을 생성하는 데 핵심적인 역할을 하며, poster_url 은 채팅창이나 상세 화면에서 시각적인 포스터 이미지를 제공하기 위해 사용된다.



[Fig 7.5] ER Diagram, Movie

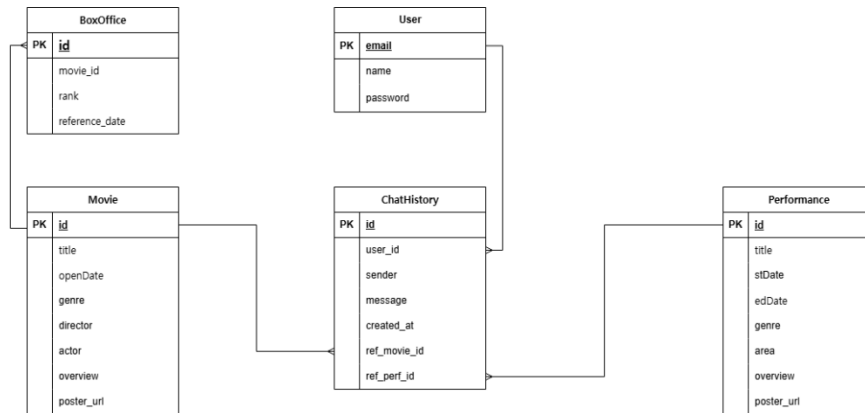
7.2.5 Performance

Performance Entity 는 영화 외에도 사용자에게 연극, 뮤지컬 등 다양한 오프라인 공연 정보를 제공하고 이를 기반으로 RAG 검색을 수행하기 위한 개체이다. Performance Entity 는 id, title, stDate, edDate, genre, area, overview, poster_url 을 attribute 로 가지며, id 가 primary key 이다. 이 attribute 들 중 title 은 공연의 제목을, stDate 와 edDate 는 공연의 시작일과 종료일을 나타내어 사용자가 관람 가능한 일정을 확인할 수 있게 하며, genre 와 area 는 각각 공연의 장르와 장소 정보를 저장하여 사용자의 취향과 위치에 맞는 공연 추천을 돕는다. 또한 overview 는 공연의 줄거리나 상세 소개 글을 담고 있어 AI 가 사용자의 질문 의도에 부합하는 공연을 검색하여 추천하는 데 핵심적인 역할을 하며, poster_url 은 채팅창이나 상세 화면에서 시각적인 포스터 이미지를 제공하여 사용자의 흥미를 유도하는 데 활용된다.



[Fig 7.6] ER Diagram, Performance

7.3 Relational Schema



[Fig 7.7] Relational Schema

8. Testing Plan

8.1 Objectives

본 테스트 계획의 목적은 LLMUSE 시스템이 SRS 에 명시된 기능 요구사항 및 성능 요구사항을 충족하는지 검증하는 데 있다. 특히 다음 항목을 중점적으로 검증한다.

- RAG 기반 검색 파이프라인의 정확성 및 안정성
- 외부 API 연동의 신뢰성
- 페이지 로딩, DB 쿼리, AI 응답 속도 등 핵심 성능 요구사항 준수 여부
- UI/UX 흐름의 매끄러움 및 사용자 경험 품질

본 테스트는 개발 단계, 통합 단계, 배포 단계, 사용자 테스트 단계에 걸쳐 수행된다.

8.2 Testing policy

LLMUSE 시스템의 테스트는 Development Testing → Integration/System Testing → Performance Testing → Release Testing → User Testing 순으로 이루어진다.

8.2.1 Development testing

Development Testing 은 기능 개발 단계에서 발생할 수 있는 오류를 조기에 발견하고 수정하는 것을 목표로 한다.

(1) Unit Testing

- Frontend: UI 컴포넌트가 독립적으로 정상 렌더링 및 이벤트 처리하는지 검증
- Backend: Service 및 Controller 계층의 함수가 의도한 대로 값을 반환하는지 검증

(2) Development-Level Integration

- Frontend ↔ Backend API 호출 과정에서 데이터 형식 충돌이 없는지 확인.
- Vector DB 와 Backend 간의 벡터 검색 로직 연동 점검.
- OpenAI/KOBIS/TMDB API Key 가 정상적으로 로딩되고 인증되는지 확인.

8.2.2 Integration and system testing

(1) Integration Testing

- Frontend ↔ Backend REST API / WebSocket 연동 테스트.
- Vector DB 와 Backend 간의 데이터 생성·조회·수정·삭제가 반영되는지 확인.

- OpenAI/KOBIS/TMDB API 호출 결과가 정상적으로 처리되는지, 호출 실패 시 Retry 로직 및 예외 처리가 정상 작동하는지 검증.

(2) System Testing

- RAG Pipeline 검증: 사용자 자연어 입력 → Vector DB 검색 → LLM 답변 생성 → UI 표시의 전체 흐름이 매끄럽게 이어지는지 평가.
- UI/UX Flow 검증: 검색 → 추천 흐름이 중단 없이 동작하는지, 데스크톱 환경에서의 사용성을 확인.

8.2.3 Performance testing

(1) 동시 접속

- 최소 10 명 이상의 동시 접속자가 있을 때 응답 지연, DB Lock, Timeout 이 없어야 한다.

(2) 응답 속도

항목	성능 기준
메인 페이지/검색 페이지 로딩	최대 2 초 이내
DB 검색 쿼리	평균 1 초 이내, 최대 3 초
OpenAI API 호출	평균 2~3 초, 최대 5 초
KOBIS/TMDB API 호출	평균 2 초, 최대 2 초

(3) 안정성

- 시스템 에러율 0.1% 이하 유지

(4) 트랜잭션 처리량

- 초당 100 건(TPS) 이상의 요청을 안정적으로 처리할 수 있어야 한다.

8.2.4 Release testing

시스템 배포 전 최종 검증을 목표로 한다.

- Alpha Version: 기본 RAG 기능, DB 적재, 검색 기능 완성 상태에서 내부 개발팀이 테스트하며 주요 버그를 집중 확인한다.
- Beta Version: 실제 사용자에게 제한적으로 배포하여 실환경에서의 응답 속도, 추천 정확도, UX 만족도를 검증하고 피드백을 수집한다.

8.2.5 User testing

실제 사용자가 시스템을 직접 사용하며 오류를 식별하는 단계로 시나리오 기반 테스트를 진행하고 사용자 만족도 조사 및 베타 피드백을 반영한다.

8.2.6 Test case

Test case 는 앞서 살펴본 performance, reliability, security 에 초점을 맞추어 설계된다. 각각의 측면에서 최소 5 개 이상의 test case 와 scenario 를 통해 본 소프트웨어에 대한 테스트를 진행하며 평가 보고서를 작성한다.

9. Development Plan

9.1 Objectives

해당 챕터에서는 시스템의 개발 환경 및 기술에 대해 설명한다.

9.2 Frontend enviornment

9.2.1 Nest.js

Next.js 는 React 기반의 서버 사이드 렌더링 프레임워크이다. 자동 코드 분할, 파일 시스템 기반 라우팅, API Routes 등의 기능을 제공한다. 본 시스템에서는 Next.js 14

버전의 App Router 를 사용하여 페이지 구조를 구성하고, 서버 사이드 렌더링을 통해 초기 로딩 속도와 SEO 를 개선한다.



[Fig 9.1] Nest.js

9.2.2 TypeScript

TypeScript 는 Microsoft 에서 개발한 JavaScript 의 상위 집합 언어로, 정적 타입 검사 기능을 제공한다. 컴파일 시점에 오류를 발견할 수 있어 코드의 안정성과 유지보수성을 향상시킨다. 본 시스템에서는 모든 React 컴포넌트와 API 인터페이스에 TypeScript 를 적용하여 타입 안전성을 확보한다.



[Fig 9.2] TypeScript

9.2.3 HyperText Markup Language (HTML)

HTML 은 웹 페이지를 위한 마크업 언어로, HTML 을 통해 제목, 단락, 목록 등 본문을 위한 구조적 의미를 나타내는 것뿐만 아니라 링크, 인용과 그 밖의 항목으로 구조적 문서를 만들 수 있는 방법을 제공받을 수 있다. 본 시스템의 웹 페이지를 구현하기 위하여 HTML 을 사용한다.



[Fig 9.3] HTML

9.2.4 Tailwind CSS

Tailwind CSS 는 Utility-First CSS 프레임워크로, 미리 정의된 클래스를 조합하여 빠르게 스타일링할 수 있다. 반응형 디자인을 위한 접두사를 제공하며, 커스텀 테마 설정을 통해 일관된 디자인 시스템을 구축할 수 있다. 본 시스템에서는 Tailwind CSS 를 사용하여 신속한 UI 개발과 반응형 레이아웃을 구현한다.



[Fig 9.4] Tailwind CSS

9.2.5 Material-UI (MUI)

Material-UI 는 Google 의 Material Design 가이드라인을 기반으로 한 React 컴포넌트 라이브러리이다. Button, Card, TextField 등 다양한 UI 컴포넌트를 제공하며, 접근성과 일관성을 보장한다. 본 시스템에서는 MUI 컴포넌트를 Tailwind CSS 와 함께 사용하여 세련되고 일관된 사용자 인터페이스를 구현한다.



[Fig 9.5] Material-UI

9.2.6 Axios

Axios는 Promise 기반의 HTTP 클라이언트 라이브러리로, 브라우저와 Node.js에서 모두 사용 가능하다. 요청 및 응답 인터셉터, 타임아웃 설정, JSON 자동 변환 등의 기능을 제공한다. 본 시스템에서는 Axios를 사용하여 Nest.js Backend와의 RESTful API 통신을 처리한다.



[Fig 9.5] Axios

9.2.4 Socket.io - client

Socket.io-client는 실시간 양방향 통신을 위한 WebSocket 클라이언트 라이브러리이다. 자동 재연결, 이벤트 기반 통신, 폴백 메커니즘 등의 기능을 제공한다. 본 시스템에서는 챗봇 인터페이스의 실시간 메시징 및 AI 응답 스트리밍을 구현하기 위해 Socket.io-client를 사용한다.



[Fig 9.6] Socket.io

9.3 Backend environment

9.3.1 ChromaDB

ChromaDB는 텍스트나 이미지 같은 비정형 데이터를 효율적으로 검색하고 관리하기 위해 설계된 오픈 소스 벡터 데이터베이스이다. 임베딩(Embedding) 모델을 통해 변환된 데이터를 벡터 형태로 저장하고, 이 벡터 간의 유사도를 계산하여 가장 관련성 높은 정보를 빠르게 찾아내는 기능을 제공한다. 따라서 우리는 본 소프트웨어에서 대규모 비정형 데이터를 저장 및 검색하고, 검색 증강 생성(RAG)과 같은 고급 AI 기능을 구현하기 위해 이를 사용할 것이다.



[Fig 9.7] ChromaDB

9.2.1 Nest.js

NestJS는 현대적인 서버 측 애플리케이션을 효율적으로 구축하기 위한 Node.js 프레임워크이다. 이는 TypeScript를 기반으로 하며, 엔터프라이즈급 애플리케이션 개발에 적합한 모듈화 및 객체 지향(OOP) 원칙을 강제하는 구조를 제공한다. 또한, Angular와 유사한 아키텍처 패턴을 사용하여 개발의 일관성과 확장성을 높인다. 따라서 우리는 본 소프트웨어의 안정적이고 확장 가능한 서버 구조를 구축하기 위해 이를 사용할 것이다.



[Fig 9.8] Nest.js

9.2.1 Github

Github 는 소프트웨어를 개발하고 Git 을 통해 버전을 관리하기 위해 사용되는 툴이다. 이를 통해 여러명의 개발자가 하나의 프로젝트를 동시에 관리하며 개발할 수 있으며, 각각의 컴포넌트들을 통합하는데 이점이 있다. 따라서 우리는 본 소프트웨어의 개발 및 버전관리를 위해 이를 사용할 것이다.



[Fig 9.9] Nest.js

9.4 Constraints

본 시스템은 이 문서에서 언급된 내용들에 기반하여 디자인되고 구현될 것이다. 이를 위한 세부적인 제약사항은 다음과 같이 나타난다.

- 기존에 널리 쓰이고 있는 기술 및 언어들을 사용한다.
- 로열티를 지불하거나 separate license 를 요구하는 기술 및 software 의 사용을 지양한다.
- 전반적인 시스템 성능을 향상시킬 수 있도록 개발한다.
- 사용자가 편리하게 이용할 수 있도록 개발한다.
- 가능한 오픈소스 소프트웨어를 사용한다.
- 시스템 비용과 유지보수 비용을 고려하여 개발을 진행한다.
- 머신러닝 및 시스템의 확장을 고려하여 개발을 진행한다.
- 시스템 자원의 낭비를 막을 수 있도록 소스 코드를 최적화한다.
- 소스 코드 작성시 유지보수에 대하여 고려하며 필요한 부분에 대해서는 주석을 통하여 이해하기 쉽도록 한다.
- 개발은 최소 윈도우 7 이상에서 이루어져야 하며 윈도우 10, 11 환경을 타겟으로 한다.
- 윈도우 10, 11 버전에서 실행한다.

9.5 Assumptions and dependencies

본 문서에 명시된 모든 시스템은 데스크톱 환경에 기반하여 디자인 및 구현되었다. 운영 체제(OS) 환경은 Windows 10 또는 11 기반으로 가정한다. 또한, 시스템의 모든 기능은 Google Chrome 브라우저의 최신 버전에서 정상적으로 실행되는 것을 전제로 한다. 따라서 명시된 OS 및 브라우저 이외의 환경(예: 다른 OS, 다른 웹 브라우저)에서는 시스템의 기능 및 지원을 보장할 수 없다.